

Relazione - Versione 9: Transformer Block

Introduzione

La Versione 9 estende la scaled dot-product attention esplicita della Versione 8 trasformandola in un Transformer block completo.

Restano invariati il corpus inglese di 440 caratteri, il vocabolario di 27 caratteri, il contesto di quattro caratteri, gli embedding addestrabili, gli embedding posizionali, lo split training/validation riproducibile, gli aggiornamenti mini-batch e la generazione autoregressiva. Il cambiamento principale è architetturale: durante il training ogni posizione predice il carattere successivo e il modello aggiunge causal self-attention, connessioni residuali, layer normalization e una rete feed-forward position-wise.

```
Caratteri
|
Identificatori interi
|
Contesto di 4 caratteri + 4 target sequenziali
|
Embedding carattere + posizione
|
Causal self-attention
|
Residual + LayerNorm
|
Rete feed-forward
|
Residual + LayerNorm
|
Logits del vocabolario a ogni posizione
|
Testo generato
```

Obiettivo della Versione 9

- riutilizzare corpus, vocabolario, contesto di quattro caratteri ed embedding a 8 dimensioni della Versione 8;
- passare da un solo target successivo a quattro target sequenziali, per esempio mode -> odel;
- applicare una causal mask per impedire a ogni posizione di vedere le posizioni future;
- aggiungere proiezione dell'output attention, connessioni residuali, LayerNorm e sottoblocco feed-forward;
- allenare tutte le posizioni usando durante la generazione solo i logits dell'ultima posizione;
- verificare forme, numero di parametri, causalità, gradienti, aggiornamenti, loss, riproducibilità e generazione.

Architettura

Il flusso completo della Versione 9 è:

```
Corpus: 440 caratteri
|
Vocabolario: 27 caratteri
|
436 esempi sequenziali contesto-target
|
Tensore input: (436, 4)
Tensore target: (436, 4)
|
Embedding caratteri: (27, 8) + posizioni: (4, 8)
|
Query / key / value: 4 x 16 per esempio
|
Matrice causal attention: 4 x 4 per esempio
|
Proiezione output attention: 16 -> 8
|
Residual + LayerNorm
|
Rete feed-forward: 8 -> 16 -> 8
|
Residual + LayerNorm
|
Logits vocabolario: (4, 27) per esempio
|
Split 80 / 20 riproducibile + mini-batch
|
Generatore autoregressivo a finestra scorrevole
```

Target sequenziali del contesto

Ogni esempio contiene ancora quattro caratteri in input, ma la Versione 9 apprende quattro previsioni allineate del carattere successivo. Con input mode, il target è odel. La posizione 0 predice o da m; la posizione 1 predice d da mo; la posizione 2 predice e da mod; la posizione 3 predice l da mode. La causal mask rende questi target coerenti con la generazione autoregressiva.

Finestre di contesto, split e mini-batch

```
esempi training = 348
esempi validation = 88
dimensione batch = 32
aggiornamenti per epoca = 11
```

Solo gli esempi di training vengono mescolati e aggiornano i parametri. La validation misura la loss senza modificare il modello. Le finestre vicine si sovrappongono, quindi questo split resta uno strumento didattico di monitoraggio e non una stima forte della generalizzazione su testo mai visto.

Parametri addestrabili

Il modello contiene 13 variabili TensorFlow addestrabili e precisamente 1264 parametri:

Embedding caratteri	27 x 8	= 216
Embedding posizionali	4 x 8	= 32
Proiezione query	8 x 16	= 128
Proiezione key	8 x 16	= 128
Proiezione value	8 x 16	= 128
Proiezione output attention	16 x 8	= 128
LayerNorm 1 scala + shift	8 + 8	= 16
Feed-forward input	8 x 16	= 128
Feed-forward output	16 x 8	= 128
LayerNorm 2 scala + shift	8 + 8	= 16
Pesi output	8 x 27	= 216

Totale		= 1264

Dettagli del Transformer block

Gli ID dei caratteri selezionano embedding addestrabili a 8 dimensioni. A ogni posizione viene aggiunto un embedding posizionale. La sequenza risultante viene proiettata in query, key e value a 16 dimensioni.

$$Q = X W_q$$

$$K = X W_k$$

$$V = X W_v$$

$$\text{scores} = Q K^T / \sqrt{d_k}$$

$$\text{masked_scores} = \text{causal_mask}(\text{scores})$$

$$A = \text{softmax}(\text{masked_scores})$$

$$H = A V$$

L'output dell'attention viene proiettato da 16 a 8 valori per posizione, così può essere sommato all'input posizionato tramite la prima connessione residuale. Segue la layer normalization. Una rete feed-forward position-wise espande 8 valori a 16, applica la non-linearità e torna a 8; una seconda residual e LayerNorm completano il blocco. Le proiezioni attention, feed-forward e vocabolario non usano bias; le due LayerNorm usano scala e shift addestrabili.

Causal mask

```
[[1, 0, 0, 0],
 [1, 1, 0, 0],
 [1, 1, 1, 0],
 [1, 1, 1, 1]]
```

Loss, training e riproducibilità

La sparse softmax cross-entropy viene calcolata in tutte e quattro le posizioni della sequenza. Le loss per posizione hanno forma (batch, 4) e la loro media diventa l'obiettivo scalare. tf.GradientTape calcola i gradienti per tutte le 13 variabili addestrabili e il gradient descent mini-batch applica manualmente gli aggiornamenti.

Learning rate: 1.0

Batch size: 32

Epoche: 100

Seed: 42

Risultato del training salvato

Loss training iniziale: 3.298020

Loss training finale: 1.915883

Loss validation iniziale: 3.299714

Loss validation finale: 2.208963

Migliore loss validation: 2.130791

Migliore epoca validation: 67

La loss di training diminuisce nettamente. La validation migliora nelle prime fasi, raggiunge il minimo all'epoca 67 e poi oscilla mentre il training continua a migliorare: un comportamento compatibile con lieve overfitting su un dataset molto piccolo. I numeri riportati sono quelli salvati nell'esecuzione corrente del notebook; tra ambienti TensorFlow possono comparire piccole differenze numeriche.

Probabilità apprese

Per il contesto mode, le probabilità finali più alte nell'esecuzione salvata sono:

spazio 0.3648

r 0.1843

d 0.0762

l 0.0639

t 0.0583

m 0.0465

Ispezione della causal attention

	m	o	d	e
m	1.0000	0.0000	0.0000	0.0000
o	0.9234	0.0766	0.0000	0.0000
d	0.2832	0.3961	0.3207	0.0000
e	0.1037	0.4917	0.2196	0.1849

Ogni riga somma a circa 1 e tutti i valori sopra la diagonale principale sono zero, confermando il vincolo causale.

Generazione autoregressiva

La generazione usa ancora una finestra scorrevole di quattro caratteri. A ogni passo l'intero Transformer block viene ricalcolato sul contesto corrente. Solo i logits dell'ultima posizione vengono trasformati in probabilità e campionati; poi il carattere più vecchio viene eliminato e il nuovo carattere viene aggiunto.

Con seed 42, cinque campioni indipendenti dal contesto mode nell'esecuzione salvata sono:

r, d, s, r, spazio

La dimostrazione da 300 caratteri inizia così:

```
moderetr win mawiom n.  
sinttl macors mod manes lexper fcongded clerning lamamp belers,  
th comakerincte crsmay rin tleleaserks pamake codvame win mexpl...
```

Controlli inclusi

- i tensori input e target hanno entrambi forma (436, 4);
- le 13 variabili addestrabili hanno le forme previste e totalizzano esattamente 1264 parametri;
- il forward produce embedding (1, 4, 8), Q/K/V (1, 4, 16), attention (1, 4, 4), output Transformer (1, 4, 8) e logits (1, 4, 27);
- la causal mask è triangolare inferiore e i pesi verso il futuro sono esattamente zero;
- le righe di attention e le distribuzioni di probabilità sommano a 1;
- tutti i gradienti sono presenti e finiti e ogni variabile addestrabile cambia durante il training;
- gli indici training e validation sono separati e le loss restano finite;
- retraining e generazione autoregressiva sono riproducibili con lo stesso seed.

La sezione di test del notebook termina con All checks passed.

Cosa insegna la Versione 9

La Versione 9 è il primo passaggio del progetto con la struttura interna di un vero Transformer block, non soltanto con un meccanismo di attention. La progressione concettuale è:

```
contesto -> embedding + posizioni
          -> Q / K / V
          -> causal attention
          -> proiezione + residual + LayerNorm
          -> feed-forward + residual + LayerNorm
          -> logits a ogni posizione
          -> loss su tutte le posizioni
```

La differenza rispetto alla Versione 8 non consiste semplicemente in più codice. Ogni nuova parte introduce un ruolo architetturale che sarà riutilizzato nei Transformer decoder-only: flusso causale dell'informazione, percorsi residuali, normalizzazione ed elaborazione non lineare position-wise.

Limitazioni

- la lunghezza del contesto resta fissa a quattro caratteri e il corpus contiene solo 440 caratteri;
- il modello usa un solo meccanismo di attention e non la multi-head attention;
- è presente un solo Transformer block, senza stacking, dropout o learning-rate schedule;
- le finestre sovrapposte possono comparire tra training e validation, quindi la validation non è un benchmark robusto di generalizzazione;
- la generazione usa i parametri dell'epoca finale invece di ripristinare automaticamente il miglior checkpoint di validation;
- l'implementazione è volutamente esplicita: il passaggio di molte variabili sta diventando verboso e potrà essere rifattorizzato in una classe o modulo nelle versioni successive.

Conclusione

La Versione 9 trasforma con successo il modello di attention della Versione 8 in un vero Transformer block causale singolo. Mantiene un ambiente character-level piccolo e ispezionabile, ma introduce i componenti strutturali necessari per il passo successivo verso un language model decoder-only più completo.

A questo punto la maggiore complessità è ancora giustificata didatticamente: ogni operazione aggiunta corrisponde a un concetto Transformer distinto. Una versione successiva potrà raccogliere le variabili in una classe o in un modulo senza nascondere la matematica già resa esplicita qui.